

복잡한 알고리즘 구현과 모듈화의 이해

R 프로그래밍 with workflow design



알고리즘 설계와 모듈화의 필요성

왜 모듈화인가?

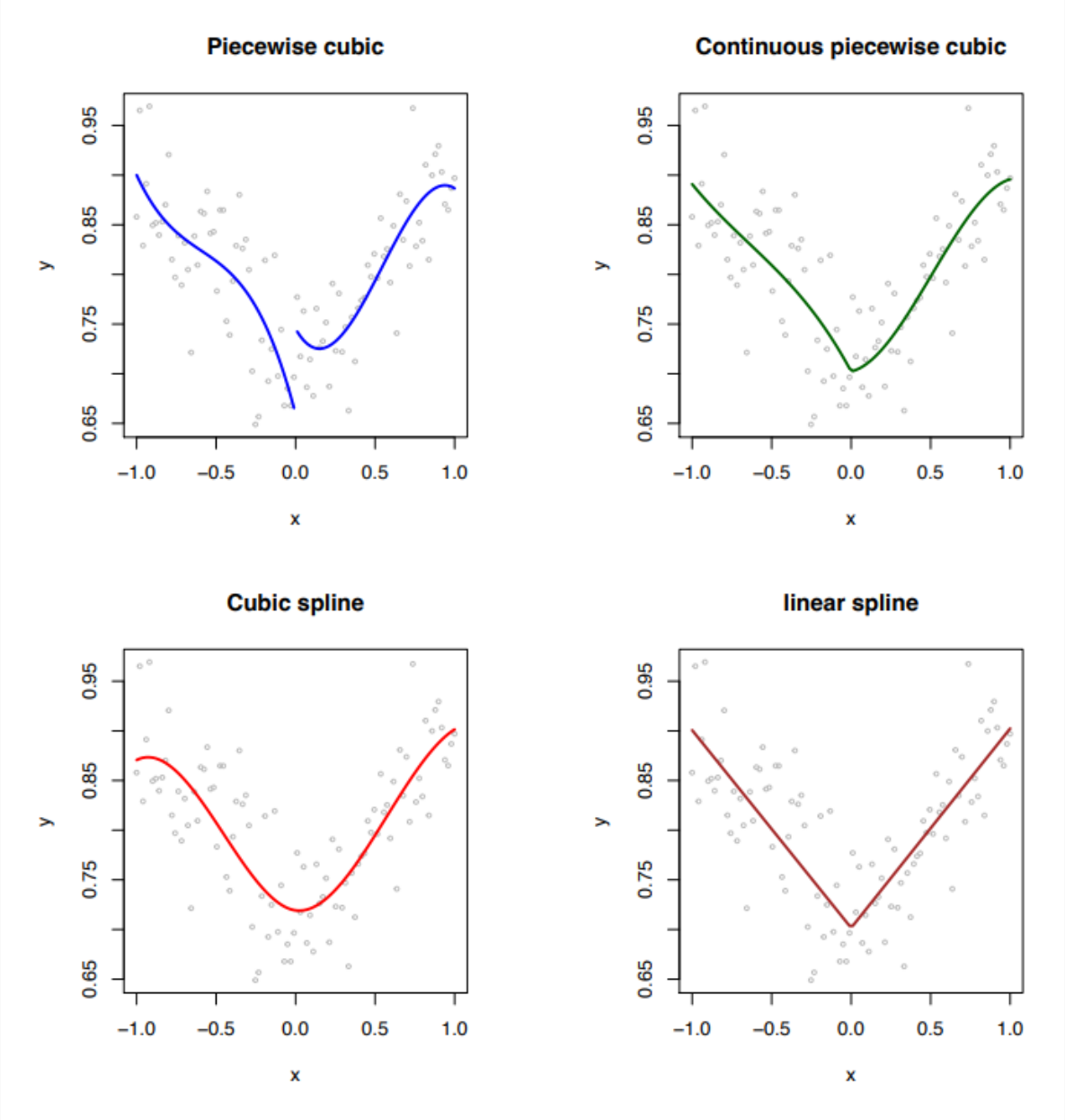
- **모듈화**: 프로그램을 작은 단위로 분할하는 개념
- 복잡한 알고리즘을 하나의 블록으로 작성하면 다음과 같은 문제를 만날 수 있음
  - 유지보수가 어려움
  - 재사용 불가
  - 디버깅이 어려움
- 해결책은 모듈화를 통한 구조적 설계
- 복잡한 알고리즘일수록 분할 정복이 핵심

스플라인 회귀 모형의 이해

스플라인 회귀란?

- 스플라인 회귀: 조건부 기댓값 $\mathbb{E}[Y|X = x]$를 스플라인 함수로 모형화
- 스플라인(Spline): 구간별로 나뉜 조각 다항식(piecewise polynomial)을 이어붙인 형태
- 핵심 특징
  - 국소적(local) 근사 가능하여 과적합 방지
  - 매듭점(knot)에서 매끄럽게 연결 (도함수 연속 조건)
- 스플라인의 구성요소
  - 차수 d (degree d): 스플라인은 각 구간에서 d차 다항식으로 구성
  - 매듭점 (knots): 스플라인 함수가 서로 다른 다항식으로 나뉘는 경계점
    - 내부 매듭점 (interior knots): 조각을 나누는 핵심 지점
    - 경계 매듭점 (boundary knots): 데이터의 시작과 끝

조각 다항식 예제


Piecewise cubic
Continuous piecewise cubic
Cubic spline
linear spline
y
x
0.65
0.75
0.85
0.95
−1.0
−0.5
0.0
0.5
1.0

스플라인 함수의 기저 표현

- 스플라인 함수는 미리 정의된 기저 함수들의 **선형결합(linear combination)**으로 표현됨
- 예를 들어 d차 스플라인 기저 $\{B_{j,d}\}_{j=1}^{J}$를 사용하면

$$g_d(x) = \sum_{j=1}^{J} \beta_j B_{j,d}(x)$$

  여기서 β_j는 추정해야 할 회귀계수이고, $B_{j,d}$는 d차 스플라인 함수의 j번째 기저함수임

- 기저 함수의 종류
  - P-스플라인 기저: 직관적 구조를 제공하고, 시각적 이해가 쉬움
  - B-스플라인 기저: 수치적으로 안정적임
- 목표
  - B-스플라인 기저 함수를 이용하여 스플라인 함수를 구현
  - 최소제곱(least squares) 방법을 통해 데이터에 적합하여 비선형 회귀 분석을 수행하는 프로그램 작성

스플라인 회귀 모형 구현을 위한 전체 설계

(1) **기저 함수 만들기**

- 입력 변수 x를 변환하기 위해 B-스플라인 기저 함수 사용
- 매듭점 시퀀스(knot sequence) 설정이 필수
- 구현 목표
  - 특정 차수 d의 B-스플라인 함수 $B_{j,d}(x)$ 계산
  - 각 x_i에 대해 다수의 기저 함수 값 계산 필요

(2) **설계 행렬 생성**

- 모든 관측값 x_i에 대해 B-스플라인 기저 함수 적용
- 설계 행렬 $G \in \mathbb{R}^{n \times J}$

$$G_{ij} = B_{j,d}(x_i)$$

- 각 행은 하나의 x_i에 대한 기저 함수 값들로 구성됨

스플라인 회귀 모형 구현을 위한 전체 설계 (cntd.)

(3) **계수 추정 (최소제곱법)**

- 최소제곱법으로 회귀 계수 $\hat{\beta}$ 추정

$$\hat{\beta} = (G^\top G)^{-1} G^\top y \quad \text{where} \quad y = (y_1, \dots, y_n) \in \mathbb{R}^n$$

(4) **예측**

- 새로운 x 값들에 대해 설계 행렬 G_{new} 생성 후 예측

$$\hat{f}(x) = \sum_{j=1}^{J} \hat{\beta}_j B_{j,d}(x)$$

(5) **시각화**

- 원래 데이터 포인트와 함께 예측된 스플라인 회귀 곡선 시각화
- 추정된 함수의 부드러운 형태 확인 가능

요약: 모듈화된 구조

단계	함수 이름 (예시)
기저 생성	`b_spline()`
행렬 구성	`create_design_matrix()`
계수 추정	`fit_spline()`
예측 수행	`predict_spline()`
시각화	`plot_spline()`

(1) 기저 함수 만들기

B-스플라인 함수

- B-스플라인이란?
  - 스플라인은 주어진 구간을 나누고 각 구간마다 다항식으로 정의되는 **조각 다항식(piecewise polynomial)** 함수
  - 가장 큰 특징은 **재귀적으로 정의 가능**하다는 점
  - 곡선의 부드러움은 차수 d에 의해 결정됨
- 매듭점 시퀀스(Knot Sequence)
  - 스플라인 함수는 **매듭점 시퀀스**를 기반으로 구간을 정의

$$T = \{t_0, t_1, \dots, t_m\}, \quad t_0 \leq t_1 \leq \dots \leq t_m$$

    - 경계 매듭점: t_0와 t_m
    - 내부 매듭점: $t_1, \dots, t_{m-1}$

B-스플라인의 차수에 따른 특징

차수 d	함수 형태	생성되는 기저 함수 개수
0	계단 함수	m
1	선형 함수	$m-1$
2	이차 함수	$m-2$
3	삼차 함수 (Cubic)	$m-3$

총 기저 함수 수 $=$ 매듭점 수 $-\,d-1$

재귀적 정의: Cox–de Boor Formula

- i번째 d차 B-스플라인 기저 함수

$$B_{i,d}(x) = \frac{x - t_i}{t_{i+d} - t_i} B_{i,d-1}(x) + \frac{t_{i+d+1} - x}{t_{i+d+1} - t_{i+1}} B_{i+1,d-1}(x)$$

- 기본 형태 ($d = 0$)

$$B_{i,0}(x) = \begin{cases} 1 & \text{if } t_i \leq x < t_{i+1} \\ 0 & \text{otherwise} \end{cases}$$

재귀적 기저 함수 (R 구현)

```r
b_spline = function(x, knots, degree, i) {
  # 0차 스플라인 기저함수의 경우
  if (degree == 0) {
    return(ifelse(knots[i] <= x & x < knots[i + 1], 1, 0))
  }

  # 재귀적인 경우
  B_i_d1 = b_spline(x, knots, degree - 1, i)
  B_i1_d1 = b_spline(x, knots, degree - 1, i + 1)
  denom1 = knots[i + degree] - knots[i]
  denom2 = knots[i + degree + 1] - knots[i + 1]
  term1 = if (denom1 == 0) 0
          else ((x - knots[i]) / denom1) * B_i_d1
  term2 = if (denom2 == 0) 0
          else ((knots[i + degree + 1] - x) / denom2) * B_i1_d1
  return(term1 + term2)
}
```

(2) 설계 행렬 생성

왜 설계 행렬이 필요한가?

- 스플라인 회귀에서는 다음과 같은 선형 모형을 사용함

$$g_d(x) = \sum_{j=1}^{J} \beta_j B_{j,d}(x)$$

- 따라서 데이터의 각 관측값 x_i에 대해 $B_{j,d}(x_i)$들을 계산해 **행렬 형태**로 저장해야 하므로 **설계 행렬(design matrix)** 필요
- 설계 행렬 G의 구조

$$G = \begin{bmatrix} B_{1,d}(x_1) & B_{2,d}(x_1) & \cdots & B_{J,d}(x_1) \\ B_{1,d}(x_2) & B_{2,d}(x_2) & \cdots & B_{J,d}(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ B_{1,d}(x_n) & B_{2,d}(x_n) & \cdots & B_{J,d}(x_n) \end{bmatrix} \in \mathbb{R}^{n \times J}$$

설계 행렬 생성 함수

```
create_design_matrix = function(x_values, knots, degree) {
  n = length(x_values)                                    # 샘플 개수
  num_basis = length(knots) - degree - 1                  # 기저 함수 수
  design_matrix = matrix(0, nrow = n, ncol = num_basis)

  for (j in 1:num_basis) {
    for (i in 1:n) {
      design_matrix[i, j] = b_spline(x_values[i], knots, degree, j)
    }
  }
  return(design_matrix)
}
```

3차 스플라인 설계행렬 생성

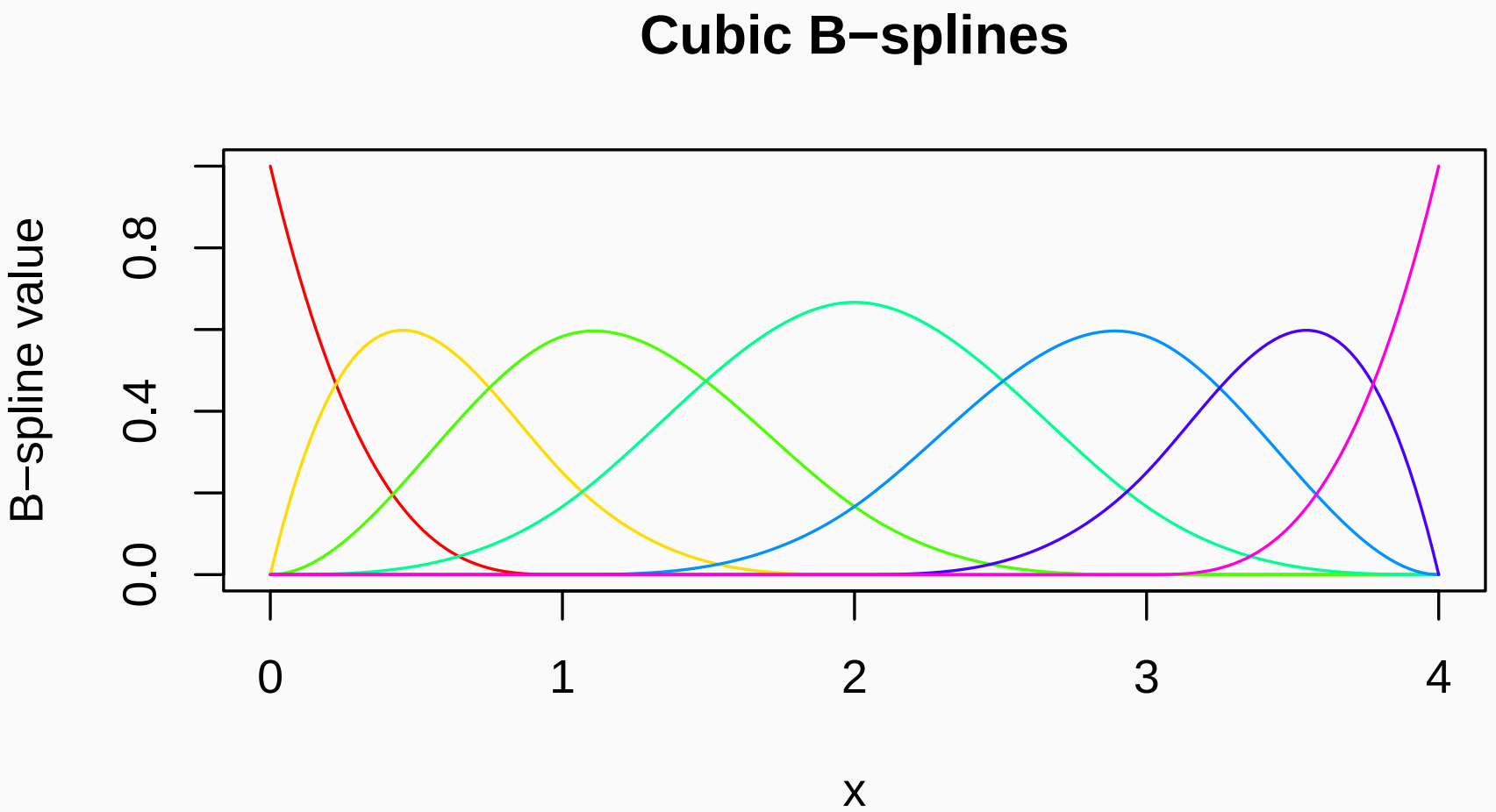
```
degree = 3
tiny = 1e-5
knots = c(rep(0 - tiny, degree), 0, 1, 2, 3, 4, rep(4 + tiny, degre
x_values = seq(0, 4, length.out = 200)

design_matrix = create_design_matrix(x_values, knots, degree)
```

- 경계 매듭점이 3차에서는 각 끝에 3회 반복되어야 함
- 수치적 안정성을 위해 경계에서 tiny를 더하고 뺌

matplot을 이용한 설계 행렬 출력

```
matplot(x_values, design_matrix, type = "l", lty = 1,
        col = rainbow(ncol(design_matrix)),
        xlab = "x", ylab = "B-spline value",
        main = "Cubic B-splines")
```



(3) 계수 추정 (최소제곱법)

문제 설정

- 데이터: $(x_i, y_i), i = 1, \ldots, n$
- B-스플라인 기저 함수 $\{B_{j,d}(x)\}_{j=1}^{J}$를 이용하여 스플라인 함수 표현

$$\begin{aligned} y_i &= g_d(x_i) + \varepsilon_i \\ &= \sum_{j=1}^{J} \beta_j B_{j,d}(x_i) + \varepsilon_i \end{aligned}$$

- 오차 제곱합 최소화

$$\min_{\beta} \|y - G\beta\|^2$$

- 최소제곱해 (closed-form solution):

$$\hat{\beta} = (G^\top G)^{-1} G^\top y$$

스플라인 최소제곱해 함수 (R 구현)

```r
fit_spline = function(x_values, y_values, knots, degree) {
  G = create_design_matrix(x_values, knots, degree)
  beta = solve(t(G) %*% G) %*% t(G) %*% y_values
  return(list(beta = beta, knots = knots, degree = degree))
}
```

(4) 스플라인 예측 함수의 구현

예측 함수가 필요한 이유

- 회귀 계수 $\hat{\beta}$를 추정한 후 새로운 입력값 x에 대한 예측값 $\hat{f}(x)$를 계산

$$\hat{f}(x) = \sum_{j=1}^{J} \hat{\beta}_j B_{j,d}(x)$$

- 즉 기존 데이터와는 다른 새로운 x 값에 대해 다시 **B-스플라인 기저 함수값을 계산**해야 함

스플라인 예측 함수 (R 구현)

```r
predict_spline = function(model, new_x) {
  G_new = create_design_matrix(new_x, model$knots, model$degree)
  y_pred = G_new %*% model$beta
  return(y_pred)
}
```

(5) 적합 결과의 시각화

왜 시각화가 중요한가?

- 데이터와 적합된 모델이 얼마나 잘 부합하는지 확인 가능
- **스플라인 회귀곡선**이 데이터의 패턴을 잘 따르고 있는지 직관적 확인 가능

```
plot_spline = function(x_values, y_values, model, grid_x) {
  y_pred = predict_spline(model, grid_x)

  data_plot = data.frame(x = x_values, y = y_values)
  spline_plot = data.frame(x = grid_x, y = y_pred)

  ggplot() +
    geom_point(data = data_plot, aes(x, y), color = "black") +
    geom_line(data = spline_plot, aes(x, y), color = "blue",
        linewidth = 1) +
    labs(title = "Fitted B-spline Regression", x = "x", y = "y") +
    theme_minimal()
}
```

스플라인 회귀 적합 예제

회귀 함수 설정

- **목표 함수**

$$f(x) = \sin(2\pi x) + \cos(4\pi x)$$

- 관측치는 위 함수에 정규 오차를 추가하여 생성

$$y_i = f(x_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2)$$

```
set.seed(123)
n = 30
x_values = sort(runif(n, 0, 1))

y_values = sin(2 * pi * x_values) +
           cos(4 * pi * x_values) +
           rnorm(n, sd = 0.2)
```

매듭점 설정 및 스플라인 적합

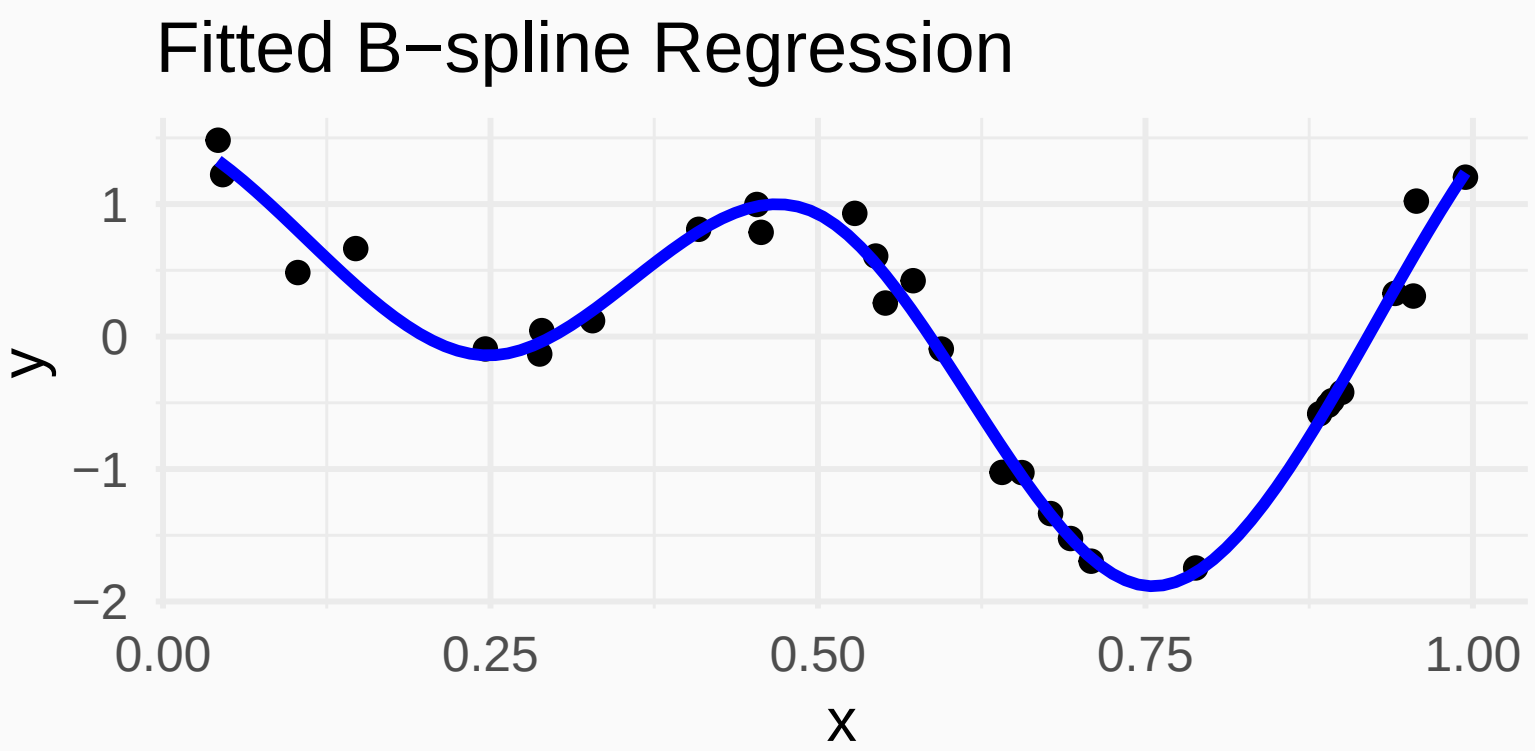
```
degree = 3
num_knots = 5
tiny = 1e-5

knots = c(rep(0 - tiny, degree),
          seq(0, 1, length.out = num_knots),
          rep(1 + tiny, degree))

model = fit_spline(x_values, y_values, knots, degree)
```

적합 결과 시각화

```
grid_x = seq(min(x_values), max(x_values), length.out = 100)
plot_spline(x_values, y_values, model, grid_x)
```



실행과 관리 전략

1. **하나의 스크립트 파일에 통합**
    - 모든 함수를 한 파일에 저장
    - 길지 않은 프로젝트에 적합
    - 주석으로 함수 목록 정리 필요

```
# 새로운 데이터에 대한 예측을 수행하는 함수
# predict_spline(model, new_x)
# - model: fit_spline()으로 학습된 스플라인 모델 (리스트)
# - new_x: 예측을 수행할 x 값 (벡터)
# - 반환값: 예측된 y 값 (벡터)
```

2. **모듈별 파일로 분할**
    - 기저 생성, 설계 행렬, 예측, 시각화 등 파일 분리 (예. `b_spline.R`)
    - `source()` 함수로 메인 스크립트에서 불러오기 (예. `source("fit_spline.R")`)